

Binary Search — Complete Patterns Sheet

How to use this sheet: Each question has the core insight ("What to do"), the approach, constraints, and a LeetCode link. When you glance at it, you should immediately recall the pattern — not just the dry run.

Pattern Index

#	Pattern	Questions
1	<u>Classic Binary Search</u>	Binary Search, Floor & Ceil, Search Insert Position, Upper & Lower Bound
2	<u>First / Last Occurrence</u>	First & Last Occurrence
3	<u>BS on Modified Array</u>	Single Element, Sqrt, Nth Root, Kth Missing
4	<u>Rotated Sorted Array</u>	Search I & II, Minimum, How Many Times Rotated
5	<u>Peak Element</u>	Find Peak I & II
6	<u>BS on Answer</u>	Koko, Smallest Divisor, Bouquets, Ship Packages, Aggressive Cows, Allocate Books, Painters, Split Array
7	<u>Two Arrays / Merge BS</u>	Median of Two Sorted Arrays, Kth Element
8	<u>2D Matrix BS</u>	Row with Max 1s, Search in 2D Matrix I & II, Peak Element II

Pattern 1 — Classic Binary Search

1. Binary Search

 [LeetCode 704](#)

Problem: Given a sorted array of n integers and a `target`, return the index if found, else return `-1`.

Example:

```
Input:  nums = [-1, 0, 3, 5, 9, 12], target = 9
Output: 4
```

Constraints: $-10^4 \leq \text{nums}[i], \text{target} \leq 10^4$ | $1 \leq n \leq 10^4$

💡 **What to do:** Standard $\text{lo}=0, \text{hi}=n-1$. Compute $\text{mid} = \text{lo} + (\text{hi}-\text{lo})/2$. If $\text{nums}[\text{mid}] == \text{target}$ return mid. If $\text{nums}[\text{mid}] < \text{target}$ go right, else go left. Return -1 if loop ends.

2. Lower Bound

[🔗 GFG / Custom](#)

Problem: Find the index of the first element $\geq$ **target** in a sorted array.

Example:

```
Input:  arr = [1, 2, 4, 6, 8], target = 5
Output: 3 (arr[3] = 6, first element  $\geq$  5)
```

Constraints: Array is sorted. Answer can be n if all elements $<$ target.

💡 **What to do:** Maintain $\text{ans} = n$. When $\text{arr}[\text{mid}] \geq \text{target}$, store $\text{ans} = \text{mid}$ and go left ($\text{hi} = \text{mid} - 1$). When $\text{arr}[\text{mid}] < \text{target}$, go right. Return ans .

3. Upper Bound

[🔗 GFG / Custom](#)

Problem: Find the index of the first element $>$ **target** in a sorted array.

Example:

```
Input:  arr = [1, 2, 4, 6, 8], target = 6
Output: 4 (arr[4] = 8, first element  $>$  6)
```

💡 **What to do:** Same as Lower Bound but condition becomes $\text{arr}[\text{mid}] > \text{target}$. Store $\text{ans} = \text{mid}$ and go left.

⚠️ **Key difference:** Lower Bound uses $\geq$, Upper Bound uses $>$.

4. Search Insert Position

Problem: Given sorted array and target, return the index to insert target so array stays sorted (or the index if already present).

Example:

```
Input:  nums = [1, 3, 5, 6], target = 2
Output: 1
```

Constraints: $1 \leq \text{nums.length} \leq 10^4$ | No duplicates.

 **What to do:** This is exactly **Lower Bound**. Find first index where $\text{nums}[i] \geq \text{target}$. That's the insert position.

5. Floor and Ceil in Sorted Array

Problem: Floor = largest element $\leq x$. Ceil = smallest element $\geq x$.

Example:

```
Input:  arr = [1, 2, 8, 10, 11], x = 5
Floor = 2,  Ceil = 8
```

 **What to do:**

- **Floor:** When $\text{arr}[\text{mid}] \leq x$, store $\text{ans} = \text{arr}[\text{mid}]$, go right. When $\text{arr}[\text{mid}] > x$, go left.
 - **Ceil:** Same as Lower Bound — find first element $\geq x$.
-

Pattern 2 — First/Last Occurrence & Count

6. First and Last Occurrence

Problem: Given sorted array (may have duplicates), find the first and last index of target . Return $[-1, -1]$ if not found.

Example:

```
Input:  nums = [5, 7, 7, 8, 8, 10], target = 8
Output: [3, 4]
```

Constraints: $0 \leq \text{nums.length} \leq 10^5$ | $-10^9 \leq \text{nums}[i] \leq 10^9$

💡 **What to do:** Run two separate binary searches.

- **First occurrence:** When `nums[mid] == target`, store `ans = mid`, go **left** (`hi = mid - 1`).
- **Last occurrence:** When `nums[mid] == target`, store `ans = mid`, go **right** (`lo = mid + 1`).

● Pattern 3 — Binary Search on Modified Sorted Array

7. Sqrt of a Number

🔗 [LeetCode 69](#)

Problem: Given integer `x`, return the integer square root (floor value). No built-in sqrt.

Example:

```
Input:  x = 8
Output: 2  (sqrt(8) ≈ 2.82, floor = 2)
```

Constraints: $0 \leq x \leq 2^{31} - 1$

💡 **What to do:** Binary search on range `[1, x]`. When `mid*mid <= x`, store `ans = mid`, go right. When `mid*mid > x`, go left. Return `ans`.

⚠️ Use `long` for `mid*mid` to avoid overflow.

8. Nth Root of M

🔗 [GFG](#)

Problem: Find the nth root of M (integer answer only). Return -1 if no integer root exists.

Example:

```
Input:  n = 3, m = 27
Output: 3  (3³ = 27)
```

💡 **What to do:** Binary search on $[1, m]$. Compute mid^n (careful of overflow — use a helper that returns 1 if exceeds m , 2 if equals, 0 if less). If equals m , return mid . If less, go right. If more, go left.

9. Find Single Element in Sorted Array

🔗 [LeetCode 540](#)

Problem: Every element appears exactly twice except one. Find that element in $O(\log n)$.

Example:

```
Input:  nums = [1, 1, 2, 3, 3, 4, 4, 8, 8]
Output: 2
```

Constraints: $1 \leq \text{nums.length} \leq 10^5$ | Array is sorted.

💡 **What to do:** The single element breaks the pair pattern. At index mid :

- If mid is **even**: normally $\text{nums}[\text{mid}] == \text{nums}[\text{mid}+1]$. If this holds, single is on the **right** → $\text{lo} = \text{mid} + 1$. Else go left.
 - If mid is **odd**: normally $\text{nums}[\text{mid}] == \text{nums}[\text{mid}-1]$. If this holds, single is on the **right** → $\text{lo} = \text{mid} + 1$. Else go left.
-

10. Kth Missing Positive Integer

🔗 [LeetCode 1539](#)

Problem: Given sorted positive integer array with no duplicates, find the k th missing positive integer.

Example:

```
Input:  arr = [2, 3, 4, 7, 11], k = 5
Output: 9
```

Constraints: $1 \leq \text{arr.length} \leq 1000$ | $1 \leq k \leq 1000$

💡 **What to do:** Missing count before index i is $\text{arr}[i] - (i+1)$. Binary search for the last index where $\text{missing} < k$. Answer = $\text{arr}[\text{hi}] + (k - \text{missing_at_hi})$.

● Pattern 4 — Rotated Sorted Array

11. Search in Rotated Sorted Array I

[LeetCode 33](#)

Problem: Sorted array rotated at unknown pivot. No duplicates. Find target index or -1.

Example:

```
Input:  nums = [4, 5, 6, 7, 0, 1, 2], target = 0
Output: 4
```

Constraints: All values are unique. $1 \leq \text{nums.length} \leq 5000$

💡 **What to do:** One half is always sorted. Check which half is sorted:

- If $\text{nums}[\text{lo}] \leq \text{nums}[\text{mid}] \rightarrow$ **left is sorted**. Check if target lies in $[\text{nums}[\text{lo}], \text{nums}[\text{mid}]$). If yes, go left. Else go right.
- Else $\rightarrow$ **right is sorted**. Check if target lies in $(\text{nums}[\text{mid}], \text{nums}[\text{hi}]$). If yes, go right. Else go left.

12. Search in Rotated Sorted Array II

[LeetCode 81](#)

Problem: Same as above but array **may have duplicates**. Return true/false.

Example:

```
Input:  nums = [2, 5, 6, 0, 0, 1, 2], target = 0
Output: true
```

💡 **What to do:** Same logic as I, but add one extra case:

When $\text{nums}[\text{lo}] == \text{nums}[\text{mid}] == \text{nums}[\text{hi}] \rightarrow$ **can't decide which side is sorted** $\rightarrow$ just do $\text{lo}++$, $\text{hi}--$ and shrink.

⚠️ Worst case $O(n)$ due to duplicates — this is expected.

13. Minimum in Rotated Sorted Array

[LeetCode 153](#)

Problem: Find minimum element in a rotated sorted array with unique elements.

Example:

```
Input:  nums = [3, 4, 5, 1, 2]
Output: 1
```

Constraints: All values unique. $1 \leq \text{nums.length} \leq 5000$

💡 **What to do:** The minimum is always in the **unsorted half**. When $\text{nums}[\text{mid}] \geq \text{nums}[\text{lo}]$, left half is sorted $\rightarrow$ min is in right half, store $\text{nums}[\text{lo}]$ as candidate, go right. Else right half is broken $\rightarrow$ min is in left or is mid, store $\text{nums}[\text{mid}]$, go left.

14. How Many Times Has the Array Been Rotated

[🔗 GFG](#)

Problem: Find the number of times a sorted array was rotated. (Equivalent to finding the index of the minimum element.)

Example:

```
Input:  arr = [4, 5, 6, 7, 0, 1, 2]
Output: 4 (minimum is at index 4)
```

💡 **What to do:** This is exactly "**Find Minimum in Rotated Sorted Array**" — just return the **index** of the minimum instead of the value. The index of the minimum = number of rotations.

● Pattern 5 — Peak Element

15. Find Peak Element I

[🔗 LeetCode 162](#)

Problem: Peak = element greater than its neighbors. Array may have multiple peaks, return any one. $\text{nums}[-1] = \text{nums}[n] = -\infty$.

Example:

```
Input:  nums = [1, 2, 1, 3, 5, 6, 4]
Output: 5 (index 5, value 6)
```

Constraints: $\text{nums}[i] \neq \text{nums}[i+1]$. Solve in $O(\log n)$.

💡 **What to do:** Move towards the **higher neighbor**. If $\text{nums}[\text{mid}] < \text{nums}[\text{mid}+1]$, peak is on the right $\rightarrow \text{lo} = \text{mid} + 1$. Else peak is on left or is mid $\rightarrow \text{hi} = \text{mid}$. When $\text{lo} == \text{hi}$, that's the peak.

16. Find Peak Element II (2D)

🔗 [LeetCode 1901](#)

Problem: In a 2D matrix, find a peak element (greater than all 4 neighbors). Return its position.

Example:

```
Input: mat = [[1,4],[3,2]]
Output: [0,1] (4 > 1 and 4 > 2)
```

💡 **What to do:** Binary search on **columns** (or rows). For mid column, find the row with the **maximum element** in that column. Check left and right neighbors. If $\text{mat}[\text{maxRow}][\text{mid}-1] > \text{mat}[\text{maxRow}][\text{mid}]$, go left. If $\text{mat}[\text{maxRow}][\text{mid}+1] > \text{mat}[\text{maxRow}][\text{mid}]$, go right. Else current is the peak.

● Pattern 6 — Binary Search on Answer (*Most Important*)

Core Idea: The answer itself lies in a range. Binary search on that range. Write a $\text{feasible}(\text{mid})$ function that checks "is this value achievable?" The search space has a monotonic property (feasible for all values $\geq$ or $\leq$ answer).

17. Koko Eating Bananas

🔗 [LeetCode 875](#)

Problem: Koko eats at speed k bananas/hour. Given pile sizes and h hours, find minimum k to finish all piles.

Example:

```
Input: piles = [3, 6, 7, 11], h = 8
Output: 4
```

Constraints: $1 \leq \text{piles}[i] \leq 10^9$ | $\text{piles.length} \leq h \leq 10^4$

💡 **What to do:** Search space = $[1, \max(\text{piles})]$. $\text{feasible}(k) = \text{total hours} = \sum \lceil \text{pile}/k \rceil \leq h$.
Find minimum k where feasible is true $\rightarrow$ store ans, go left.

18. Find the Smallest Divisor Given a Threshold

[LeetCode 1283](#)

Problem: Find the smallest divisor such that sum of $\lceil \text{nums}[i]/\text{divisor} \rceil \leq \text{threshold}$.

Example:

```
Input:  nums = [1, 2, 5, 9], threshold = 6
Output: 5
```

Constraints: $1 \leq \text{nums}[i] \leq 10^6$ | $\text{nums.length} \leq \text{threshold} \leq 10^6$

💡 **What to do:** Search space = $[1, \max(\text{nums})]$. $\text{feasible}(d) = \sum \lceil \text{nums}[i]/d \rceil \leq \text{threshold}$.
Minimise d . Same pattern as Koko.

19. Minimum Number of Days to Make M Bouquets

[LeetCode 1482](#)

Problem: Make m bouquets, each needing k adjacent bloomed flowers. Bloom day of each flower given. Find minimum days.

Example:

```
Input:  bloomDay = [1,10,3,10,2], m = 3, k = 1
Output: 3
```

Constraints: If $m * k > n$, return -1.

💡 **What to do:** Search space = $[1, \max(\text{bloomDay})]$. $\text{feasible}(\text{day}) = \text{count bouquets possible when flowers with bloomDay} \leq \text{day are bloomed}$. Count consecutive groups of k .
Check if total bouquets $\geq m$.

20. Capacity to Ship Packages Within D Days

[LeetCode 1011](#)

Problem: Ship weights in order within D days. Find minimum ship capacity.

Example:

```
Input: weights = [1,2,3,4,5,6,7,8,9,10], days = 5
Output: 15
```

Constraints: $1 \leq \text{weights}[i] \leq 500$ | $1 \leq D \leq 50000$

💡 **What to do:** Search space = $[\max(\text{weights}), \text{sum}(\text{weights})]$. $\text{feasible}(\text{cap})$ = simulate days needed greedily. If adding next weight exceeds cap, start new day. Check if days $\leq D$.

21. Aggressive Cows

🔗 [SPOJ](#) / [GFG](#)

Problem: Place c cows in n stalls such that the **minimum distance** between any two cows is **maximised**.

Example:

```
Input: stalls = [1, 2, 4, 8, 9], cows = 3
Output: 3
```

💡 **What to do:** Sort stalls. Search space = $[1, \text{stalls}[n-1] - \text{stalls}[0]]$. $\text{feasible}(\text{minDist})$ = greedily place cows — place next cow only if distance from last placed $\geq \text{minDist}$. Check if we can place all c cows. Maximise $\rightarrow$ find last value where feasible is true.

22. Allocate Books (Book Allocation)

🔗 [GFG](#)

Problem: Allocate books to m students such that each student gets contiguous books and the **maximum pages** assigned is **minimised**. Each student must get at least one book.

Example:

```
Input: books = [12, 34, 67, 90], students = 2
Output: 113
```

Constraints: $m \leq n$. Return -1 if impossible.

💡 **What to do:** Search space = $[\max(\text{books}), \text{sum}(\text{books})]$. $\text{feasible}(\text{maxPages})$ = greedily assign — keep adding to current student until exceeds maxPages, then give to next student. Check if students needed $\leq m$. Minimise.

23. Painter's Partition

 [GFG](#)

Problem: k painters, each paints contiguous boards. Time to paint = length. Minimise the **maximum time** any painter takes.

Example:

```
Input:  boards = [10, 20, 30, 40], k = 2
Output: 60
```

💡 **What to do: Identical to Book Allocation.** Search space = $[\max(\text{boards}), \text{sum}(\text{boards})]$.
 $\text{feasible}(\text{maxTime})$ = count painters needed greedily. Minimise.

⚠️ Recognise same pattern, same code structure.

24. Split Array Largest Sum

 [LeetCode 410](#)

Problem: Split array into k non-empty subarrays to minimise the **largest subarray sum**.

Example:

```
Input:  nums = [7, 2, 5, 10, 8], k = 2
Output: 18
```

Constraints: $1 \leq \text{nums}[i] \leq 10^6$ | $1 \leq k \leq \min(50, n)$

💡 **What to do: Identical to Painter's Partition / Book Allocation.** Same code, different words. Search space = $[\max(\text{nums}), \text{sum}(\text{nums})]$. Minimise.

⚠️ These 3 questions (Allocate Books, Painter's Partition, Split Array) are the **same problem**.

● Pattern 7 — Binary Search on Two Arrays

25. Median of Two Sorted Arrays

 [LeetCode 4](#)

Problem: Find the median of two sorted arrays in $O(\log(m+n))$.

Example:

```
Input:  nums1 = [1, 3], nums2 = [2]
Output: 2.0
```

Constraints: $0 \leq m, n \leq 1000$ | Must be $O(\log(m+n))$.

💡 **What to do:** Binary search on the **smaller array**. Partition both arrays such that left half has $(m+n)/2$ elements. Ensure $\text{maxLeft1} \leq \text{minRight2}$ and $\text{maxLeft2} \leq \text{minRight1}$. Adjust partition using lo/hi .

⚠️ **Hardest problem in this list.** Handle edge cases: empty partitions (use $-\infty / +\infty$). Always binary search on the smaller array.

26. Kth Element of Two Sorted Arrays

[GFG](#)

Problem: Find the kth smallest element in the union of two sorted arrays.

Example:

```
Input:  arr1 = [2, 3, 6, 7, 9], arr2 = [1, 4, 8, 10], k = 5
Output: 6
```

💡 **What to do:** Similar to Median problem. Binary search on smaller array. Partition such that left side has exactly k elements total. Find the maximum of the left side.
Alternatively: generalisation of median (median is kth element where $k = (m+n)/2$).

🟦 Pattern 8 — Binary Search in 2D Matrix

27. Row with Maximum Number of 1s

[GFG](#)

Problem: In a binary matrix where each row is sorted, find the row with maximum 1s.

Example:

```
Input:  mat = [[0,1,1,1],
               [0,0,1,1],
               [1,1,1,1],
               [0,0,0,0]]
Output: 2
```

💡 **What to do:** For each row, find the **first occurrence of 1** using binary search (Lower Bound of 1). Count of 1s = $n - \text{firstIndex}$. Track row with maximum count. Total = $O(m \log n)$.

28. Search in a 2D Matrix I

🔗 [LeetCode 74](#)

Problem: Matrix where each row is sorted AND first element of each row > last element of previous row. Find target.

Example:

```
Input:  matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3
Output: true
```

Constraints: $m, n \leq 100$

💡 **What to do:** Treat the entire matrix as a **1D sorted array**. Binary search with $lo=0, hi=m*n-1$. Map $mid \rightarrow \text{matrix}[mid/n][mid\%n]$. Standard binary search.

29. Search in a 2D Matrix II

🔗 [LeetCode 240](#)

Problem: Matrix where each row is sorted left to right AND each column is sorted top to bottom (but rows are NOT linked). Find target.

Example:

```
Input:  matrix = [[1,4,7,11],[2,5,8,12],[3,6,9,16],[10,13,14,17]], target = 5
Output: true
```

💡 **What to do:** Start at **top-right corner** (or bottom-left). If $\text{mat}[r][c] == \text{target} \rightarrow$ found. If $\text{mat}[r][c] > \text{target} \rightarrow$ go left ($c--$). If $\text{mat}[r][c] < \text{target} \rightarrow$ go down ($r++$). $O(m+n)$.

⚠️ This is NOT standard binary search — it's a two-pointer elimination approach on the matrix structure.

🧠 Master Cheat Sheet — Identify the Pattern Fast

Signal in Problem	Pattern
Sorted array, find a value	Classic BS
"First / last position" or duplicates	First/Last Occurrence
"Find without sqrt/pow" on single value	BS on value range
"Rotated", "pivoted", "unknown rotation"	Rotated Array BS
"Peak", "local maximum"	Peak Element
"Minimum maximum" or "Maximum minimum"	BS on Answer
"Minimum days/speed/capacity to achieve X"	BS on Answer
"Two sorted arrays", "kth element", "median"	Two Array BS
"Sorted rows/columns", matrix search	2D Matrix BS

⚠️ Common Mistakes to Avoid

1. `mid = (lo+hi)/2` — causes integer overflow. Always use `mid = lo + (hi-lo)/2`
 2. **Infinite loop** — wrong boundary update. When going left: `hi = mid - 1`. Going right: `lo = mid + 1`. Only use `hi = mid` in peak-style problems.
 3. **BS on Answer — wrong search space** — lo and hi must cover ALL possible answers including edge cases.
 4. **Rotated Array II duplicates** — forgetting the `lo++, hi--` case when `nums[lo]==nums[mid]==nums[hi]`.
 5. **Median of Two Arrays** — not binary searching on the smaller array leads to index out of bounds.
 6. **BS on Answer direction** — minimise answer → find first true (store ans, go left). Maximise answer → find last true (store ans, go right).
-